

Ayudantía Solidaria Pre-certamen 2

Ignacio González Bravo

ELO-329: Diseño y Programación Orientado a Objetos.

Universidad Técnica Federico Santa María

24 de junio de 2025

1. Mencione al menos 5 elementos que componen el stack de Android.

2. Implemente el destructor de la siguiente clase:

```
1  class Airplane{  
2      public:  
3          Airplane();  
4          ~Airplane();  
5      private:  
6          double * speed;  
7          int * forces = new int[10];  
8  };
```

3. En el siguiente código se han declarado varias funciones miembro dentro de la clase MiClase con modificadores const.

```
1  class MiClase{
2      public:
3          void add1(int *pinput, int x);
4          void add2(int *pinput, int x) const;
5          void add3(int * const pinput, int x);
6          void add4(const int *pinput, int x);
7          void add5(const int * const pinput, int x) const;
8      private:
9          int * px = new int(5);
10 };
```

Indique cual de las siguientes implementaciones de los métodos generará errores. Indique la razón en caso de que genere un error.

a)

```
1  void MiClase::add1(int * pinputm int x){
2      this -> px += *pinput + x;
3      std::cout << *px << std::endl;
4  }
```

b)

```
1  void MiClase::add2(int * pinput, int x) const{
2      this->px += *pinput + x;
3      std::cout << *px << std::endl;
4  }
```

c)

```
1 void MiClase::add3(int * const pinput, int x){  
2     this->px += *pinput + x;  
3     std::cout << *px << std::endl;  
4 }
```

d)

```
1 void MiClase::add4(const int * pinput, int x){  
2     this -> px += *pinput + x;  
3     std::cout << *px << std::endl;  
4 }
```

e)

```
1 void MiClase::add5(const int * const pinput, int x)  
2     const{  
3     this-> px += *pinput + x;  
4     std::cout << *px << std::endl;  
5 }
```

4. Identifique todos los usos de const que se podrían omitir en el siguiente código:

```
1  #include <iostream>
2  using namespace std;
3
4  class Employee{
5  public:
6      Employee(const float s, const String n, const Employee
7          * boss ) {
8          salary = s;
9          name = n;
10         this->boss = boss;
11     }
12
13     Employee & operator=(const Employee & other ) const {
14         Employee * result_emp = new Employee(other.salary,
15             other.name, other.boss);
16         return *result_emp;
17     }
18
19     void print_information( ) const {
20         cout << "Empleado con nombre " << name << " tiene
21             un salario "
22             << salary;
23     }
24
25 private:
26     float salary;
27     std::String name;
28     Employee * boss;
29 }
```

5. Sabiendo que el compilador ubica la variable **i** en la dirección 2345 u **j** 2357 ¿Qué debería imprimir el siguiente código? Especifique según cada línea.

```
1      int i[3] = {5, 7, 11};
2      int * j = i;
3      int * &k = j;
4
5      cout << "i = " << i << endl; // (1)
6      cout << "&j = " << &j << endl; // (2)
7      cout << "k = " << k << endl; // (3)
8      cout << "*k = " << *k << endl; // (4)
9      cout << "*(j+2)= " << *(j+2) << endl; // (5)
```

7. En el contexto de la Ingeniería de Software, describa qué es un caso de uso y qué es una historia de usuario. Indique al menos dos diferencias entre casos de uso e historias de usuarios.

8. Para cada segmento de código indique si es válido (compila sin errores) o no, en caso de tener error, indique el error.

6

9. Complete las declaraciones de las clases MiClase y Observer para lograr consistencia con el siguiente código:

```
1 MiClase miClase;  
2 Observer obs;  
3 QObject::connect(&miClase, SIGNAL(puntoDeControl()), &obs, SLOT  
    (imprimaTiempo()));
```

10. Considerando las declaraciones de clases dadas, indique que aparecerá por consola al ejecutar el código con y sin métodos virtuales.

```
1  class A {
2      public:
3          virtual void foo() { cout << A << endl; }
4  };
5  class B: public A {
6      public:
7          virtual void foo() { cout << B << endl; }
8  };
9  class C: public B {
10     public:
11         virtual void foo() { cout << C << endl; }
12 };
13
14 B b;
15 A *a=&b;
16 a ->foo(); // (1)
17 C c;
18 A aa=c;
19 aa.foo(); // (2)
20 a=&c;
21 B *bb = (B*) a;
22 bb->foo(); // (3)
```